The co-simulation

The firmware in `../firmware/` is a black box: it boots, DMAs some IQ around, and produces samples. This half opens it.

Co-simulation here means two things running against the same stimulus and being compared sample for sample:

- the **real firmware**, executing on the VSPA instruction-set simulator, and
- a **bit-accurate Python model** of the same DSP.

The model is not a floating-point convenience re-implementation. It reproduces the VSPA's arithmetic exactly — SM16 sign-magnitude encoding, the AU's rounding, the DIF FFT's bit-reversed output order — so when the two agree, they agree *to the last bit*, and "the firmware is correct" becomes something you can assert rather than hope.

It has a second job, and in practice the more useful one: **taking the DFE reference app apart into its building blocks** so you can see what each one does.

The parts

`native/`	the VSPA ISS, driven **in-process via ctypes** on `libvspa2_16au.so`. No server, no subprocess, no Wine. `NativeModel` is the device; `NativeSession` wraps load/boot/run.
`dfe/`	drives the firmware itself. `DfeSession` does the boot handshake, mode config and per-symbol injection the DFE expects. `NativeKernelRunner` drives a single-stage kernel.
`vspa_model/`	the bit-accurate Python model — the **independent oracle**. `arith` (SM16, rounding), `fft`, `filter`, `nco`, `qam`, `nr5g`, `chains`, …
`kernels/`	one ELF per DSP stage — see `kernels/README.md`
`scripts/`	the verification tiers, the BER co-sim, the tutorial figures
`tests/`	unit tests (vector parsing, IPPU execution)
`tools/`	the silicon anchor: MD5s captured from a **real LA9310**

Everything here is ours — none of the 61 `NXP Confidential` files live in this half; they are all under `../firmware/`.

Two ways to run the DSP

Whole firmware — boot the real image, hand it one frequency-domain symbol, read back what the chip would have transmitted:

```
from dfe.session import DfeSession
d  = DfeSession("firmware/eld/dfe-la9310-1t2r-25m30k.eld", nsc=61)
td = d.process_symbol(grid.symbol_bytes(0))   # 1024 complex samples, natural order
```

`process_symbol` takes NXP's wire format (NSC complex REs, int16 I/Q, little-endian, two's complement) and returns natural-order time-domain samples at the firmware's own fixed-point scale — doing the SM16 and bit-reversal conversions on the way in and out.

One stage at a time — the building blocks. Each stage is its own ELF that links the *same* `lib_DFE_Ref_vspa2.elb` the firmware links, so you are stepping through the real iFFT and the real filters, not a copy:

```
from dfe.kernel_runner import NativeKernelRunner, Port
tx = NativeKernelRunner("cosim/kernels/build/compute_dfe_tx_vspa2_16au.eld",
                        inputs=[Port("sc", "input_sc_packed", 61)],
                        outputs=[Port("td", "work_buf_out", 1024)])
td = tx.compute({"sc": subcarriers})["td"]
```

That is what makes the link-level BER sweep possible: run TX, put a channel in the middle, run RX, count bit errors (`scripts/dfe_cosim.py`).

How data gets into the simulator

Not through files. The host writes the input buffer and reads the output buffer **directly out of DMEM, by ELF symbol**.

Two reasons, and both are worth knowing before you write your own harness:

1. The simulator runs *in this process*. Its memory is simply there. Files were only ever a way to talk to a separate `runsim` process.
2. The native ISS services exactly **three** syscalls — `write`, `clock`, `time`. **`open` and `read` return −1.** A harness that `fopen`s its vectors gets a NULL `FILE*` and quietly computes on garbage.

And one subtlety that is silent when you get it wrong: the kernels' I/O buffers are C globals, so they live in `.bss`, and **crt0 zeroes `.bss` at boot — after the image is loaded**. A host write between `load_elf` and `boot` is therefore wiped, and the kernel returns a clean, plausible, entirely wrong answer. The runner writes its inputs at a **breakpoint on `main`**: past crt0, before the kernel has looked at them.

What the model had to get right

These are the conversions that decide whether a comparison means anything. All four were found the hard way; `../docs/TESTING.md` has the full story.

- **SM16** — the assembly kernels use sign-magnitude, not two's complement. Read them as two's complement and *most* samples still look fine, which is what makes it dangerous.
- **Bit-reversal** — the DIF iFFT emits its bins out of order; the IPPU (a separate engine, with its **own memory array**) puts them back.
- `buf_stat` carries the symbol size; get it wrong and you read the right samples from the wrong place.
- `NSC <= 2N/3` — a hard limit on the subcarrier count that nothing enforces.

Running it

From the repo root:

```
make verify        # tiers 1-5
make verify-full   # tiers 1-9, including the silicon anchor
make cosim         # the BER-vs-SNR sweep
```